PIXCRAWLER

PROJECT DOCUMENTATION

Python-Based Image Dataset Builder

Automated Discovery, Download, and Organization of Images from Multiple Search Engine Providers

Development Team **Team Members**

Amr Muhamed Fathy
Marwan Hatem Ahmed
Aya Alaa Hamed
Mohamed Hassan Abdelmonem
Osama Mohamed Ali Abdel-Gaffar
Ganna Tarek Mohamed Mansour

NOV 30, 2025

CONTENTS

Project Overview 3
Project Planning 4
Stakeholder Analysis 6
System Architecture 8
Database Design 11
API Design 12
Processing Pipeline 14
Validation & Data Quality Strategy 16
Storage & Performance Optimization 17
Scalability & Infrastructure 17
Security & Compliance 18
UX | UI Design 19
CI/CD & DevOps 24

PROJECT OVERVIEW

PixCrawler is an automated, intelligent platform for building high-quality image datasets for machine learning, research, and computer-vision applications. It streamlines the entire lifecycle of dataset creation from collecting images across the web, validating metadata, processing and optimizing assets, to storing them efficiently in scalable **cloud environments**.

The system removes the manual overhead typically required in dataset preparation by offering **automated crawling, deduplication, filtering, quality checks, labeling assistance, and multi-tier storage optimization.** This enables teams to generate reliable, production-ready datasets in a fraction of the time.

PixCrawler is built for ML engineers, data scientists, research teams, and organizations that require **continuous dataset updates or large-scale automated data ingestion**. With its modular pipeline, strong validation strategy, and fault-tolerant architecture, the platform can scale to millions of images while maintaining performance and compliance with retention policies.

Ultimately, **PixCrawler** aims to become **a standardized, end-to-end solution for dataset creation and management** combining automation, transparency, and efficiency to accelerate machine learning development.

PROJECT PLANNING

1. PROJECT OVERVIEW

PixCrawler is a **SaaS** platform designed to **automate the creation of high-quality image datasets** for machine learning, research, and enterprise applications. It **transforms keyword-based queries into fully processed, validated, deduplicated, and organized datasets** with minimal user effort

2. PROJECT OBJECTIVES

Enable **automated discovery, validation, organization, and storage of large-scale image datasets.**

Support **high-performance parallel processing** for fast dataset generation.

Provide a user-friendly dashboard for job creation, tracking, and dataset downloads.

Maintain a **scalable architecture supporting thousands of simultaneous jobs.**

Ensure enterprise-grade security, reliability, and storage lifecycle optimization.

PROJECT PLANNING

3. SCOPE

In Scope:

-Image crawling, downloading, validation, deduplication, organization.
-Hot and warm dataset compression and storage.
-Web dashboard for user management and dataset workflows.
-API endpoints for automation and developer integration.
-Cloud-first deployment with compute scaling.

Out of Scope:

-Video dataset generation.
-On-device processing agents.
-Real-time labeling tools.

4. SUCCESS METRICS

-**Performance**: Generate 1,000 images in under 2–3 minutes.
-**Reliability**: 95% system uptime.
-**Data Quality**: 98% duplicate detection accuracy.
-**Scalability**: Handle 35 concurrent dataset jobs.

PROJECT PLANNING

5. RISKS & CONSTRAINTS

-External engine rate limits.
-Network variability.
-Storage optimization required.
-Abuse prevention via rate limiting and validation.

STAKEHOLDER ANALYSIS

1. PRIMARY STAKEHOLDERS

Stakeholder	Role	Needs
ML Researchers	Use datasets for experiments	High-quality, validated images; metadata; reproducibility
Data Scientists	Build prototypes	Fast dataset creation; scalable pipelines
AI Startups	Product development	Automation; API; scalability
Developers	Integration	REST API; docs; SDKs

STAKEHOLDER ANALYSIS

2. SECONDARY STAKEHOLDERS

Stakeholder	Role	Needs
Cloud Administrators	Infra management	Autoscaling; monitoring; secure storage
Business Owners	Monetization & strategy	Usage analytics; billing; growth

3. INTERNAL STAKEHOLDERS

Team	Responsibilities
Backend Team	APIs, workers, pipelines
Frontend Team	Dashboard, UX/UI
DevOps	CI/CD, monitoring, scaling
Security Team	Authentication, privacy

SYSTEM ARCHITECTURE

1. OVERVIEW

PixCrawler is designed as a **modular, scalable, and fault-tolerant platform** for automated image dataset creation. Its architecture separates responsibilities into distinct layers and components to ensure high performance, maintainability, and extensibility.

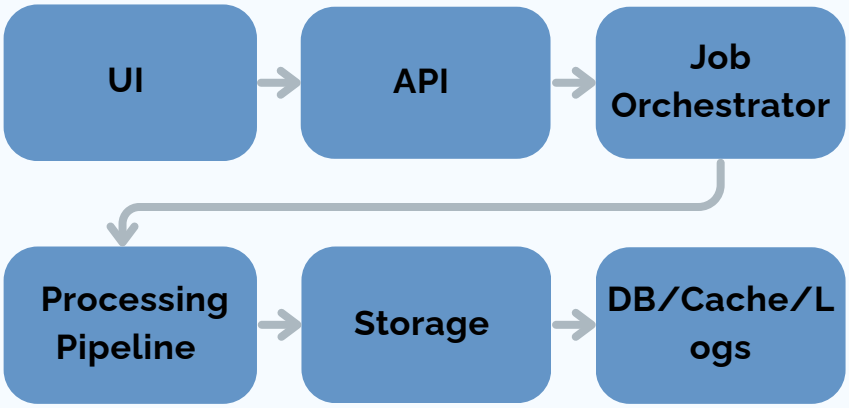
The system architecture can be summarized into the following layers:

-**User Interface (UI)**: Web dashboard for dataset configuration, job monitoring, and visualization of statistics.
-**API Layer**: FastAPI backend that handles requests, validation, authentication, and coordination of dataset jobs.
-**Job Management & Orchestration**: Celery task queue and job orchestrator manage asynchronous execution of dataset building workflows.
-**Processing Pipeline**: Modular image processing stages, including keyword generation, URL discovery, image download, validation, labeling, deduplication, and dataset assembly.
-**Storage System**: Multi-tier cloud storage (hot, warm, cold) optimized for performance and cost efficiency.
-**Data Persistence**: PostgreSQL database (via Supabase) for metadata, Redis for caching, and log storage for auditing.

SYSTEM ARCHITECTURE

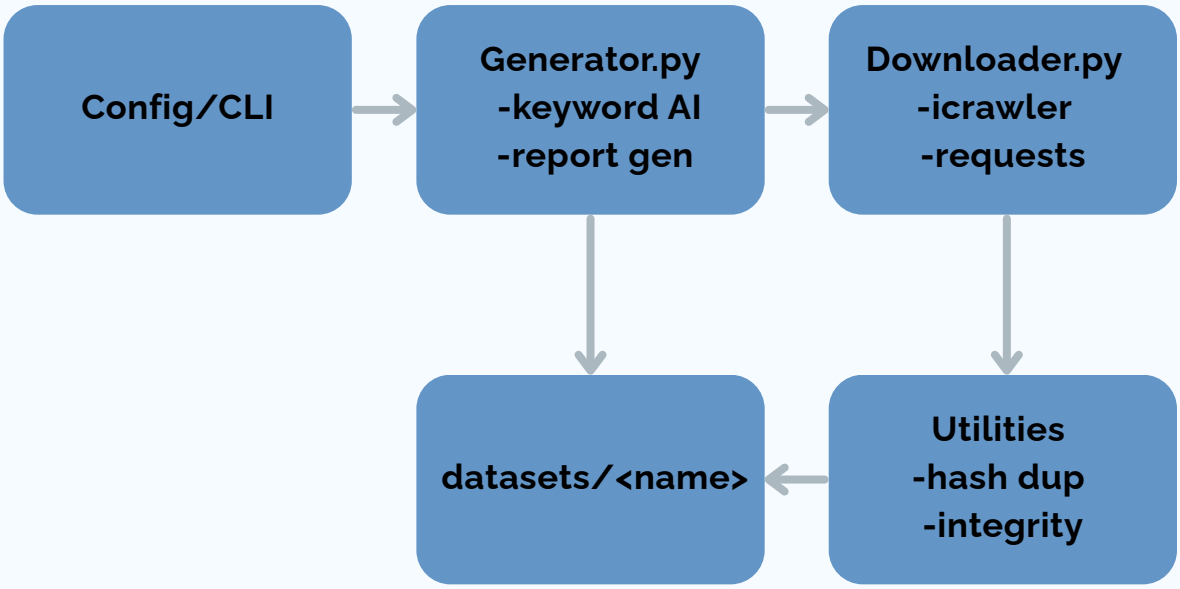
2. COMPONENT DIAGRAM

HIGH-LEVEL ARCHITECTURE:



SHOWS MAJOR LAYERS AND COMPONENTS OF THE SYSTEM, THEIR RESPONSIBILITIES, AND HOW THEY INTERACT.

THE FOLLOWING DIAGRAM ILLUSTRATES THE HIGH-LEVEL ARCHITECTURE AND DATA FLOW OF **PIXCRAWLER**:



SYSTEM ARCHITECTURE

3. PROCESSING PIPELINE ARCHITECTURE

-**Phase 1**: Keyword Generation – AI-powered keyword expansion and prioritization
-**Phase 2**: URL Discovery – Multi-engine search, aggregation, and deduplication of image URLs
-**Phase 3**: Image Download & Validation – Parallel downloading with multi-tier validation (fast, medium, slow)
-**Phase 4**: Dataset Organization & Labeling – Categorization, labeling, and manifest creation
-**Phase 5**: Storage & Distribution – Multi-tier storage, CDN distribution, and cleanup

4. TECHNOLOGY STACK

-**Frontend**: Next.js 15, TypeScript
-**Backend**: FastAPI, Celery (async job processing)
-**Database**: Supabase PostgreSQL (shared between frontend and backend), Redis cache
-**Storage**: Azure Blob Storage (Hot, Cool, Archive tiers)
-**Other Tools**: Drizzle ORM (frontend), SQLAlchemy/asyncpg (backend), logging/metrics systems

SYSTEM ARCHITECTURE

5.DESIGN PRINCIPLES

Modularity – Each processing stage is independent for easy maintenance and scalability

Scalability – Supports millions of images via distributed workers

Fault Tolerance – Jobs are monitored, retried on failure, and progress is tracked

Efficiency – Optimized storage tiering, parallel processing, and caching

Transparency & Auditability – Logs, validation reports, and metrics are captured for each dataset

DATABASE DESIGN

1.DESIGN PRINCIPLES

PixCrawler uses PostgreSQL as the core database and Redis for caching and task routing.

2. CORE ENTITIES

Users: id, email, password_hash, created_at, subscription_tier
Datasets: id, user_id, name, status, category_count, total_images, created_at
Jobs: id, dataset_id, phase, progress, started_at, finished_at
Images: id, dataset_id, path, resolution, hash_phash, hash_dhash, validation_status
Storage Records: id, dataset_id, tier, size, created_at

DATABASE DESIGN

3. ERD

Users (1—N) Datasets → (1—N) Images & Jobs → (1—N) Logs

4. INDEX STRATEGY

B-tree on user_id
Hash index on pHash/dHash
Partial indexes on validation_status
Partitioning for logs

5. RETENTION

Temp job data: 7 days
Warm storage → archive: 365 days
Logs archived after 180 days

API DESIGN

PixCrawler exposes a RESTful API built using **FastAPI**, following a clean, consistent, and well-structured endpoint strategy. The API is designed to be predictable, versioned, rate-limited, and fully typed using Pydantic models.

1.DESIGN PRINCIPLES

Versioned endpoints (/api/v1/...)
Consistent resource naming
Strong validation layer
Clear error codes (400, 404, 422, 429, 500)
Pagination for list endpoints
Auth-protected using JWT or Supabase Auth
Rate limiting per user/tenant

DATABASE DESIGN

2.MAIN ENDPOINTS

Datasets

GET /api/v1/datasets/ — list datasets (paginated)
POST /api/v1/datasets/ — create dataset
GET /api/v1/datasets/{id} — get dataset details
DELETE /api/v1/datasets/{id} — delete dataset

Build Jobs

POST /api/v1/datasets/{id}/build — start dataset build
GET /api/v1/jobs/{job_id} — track job progress
POST /api/v1/jobs/{job_id}/retry — retry failed phase
POST /api/v1/jobs/{job_id}/abort — abort running job

Artifacts

GET /api/v1/datasets/{id}/download — download zip/7z
GET /api/v1/datasets/{id}/metadata — retrieve metadata

Admin / System

GET /api/v1/system/health — health checks
GET /api/v1/system/limits — rate & quota details

3.REQUEST/RESPONSE MODELS

-DatasetCreate
-KeywordRule
-JobStatus
-ArtifactManifest
-ValidationReport

DATABASE DESIGN

4. ERROR HANDLING STANDARD

-DatasetNotFoundError
-InvalidRuleError
-RateLimitExceeded
-JobFailedError

PROCESSING PIPELINE

PixCrawler uses a modular multi-phase pipeline designed for reliability, transparency, and scalability.

1.PIPELINE PHASES

PHASE 1 — KEYWORD EXPANSION

-AI-assisted keyword expansion
-Dedup & prioritization
-Rule-based filters

PHASE 2 — URL DISCOVERY

-Search across multiple engines
-HTML/JSON parsing
-URL deduplication
-Domain filtering, MIME checks

PHASE 3 — IMAGE DOWNLOADING

-Parallel downloading (async workers)
-Temp storage
-Fast validation (size, format, corruption)

PROCESSING PIPELINE

1.PIPELINE PHASES

PHASE 4 — VALIDATION & PROCESSING

-Resolution checks
-Blur detection
-Duplicate hashing
-Unsafe content detection
-Metadata extraction

PHASE 5 — DATASET ASSEMBLY

-Folder structuring
-Category mapping
-Manifest creation (JSONL/YAML)
-Statistics generation

PHASE 6 — ARCHIVAL & STORAGE TIERING

-ZIP/7z generation
-Move to hot / warm / cold storage
-Cleanup temporary file

2.ORCHESTRATION

MANAGED BY:

-Celery workers
-Job Orchestrator
-Safe retries & checkpointing

VALIDATION & DATA QUALITY STRATEGY

1.MULTI-LAYER VALIDATION

LAYER 1 — FAST VALIDATION

-File type
-Size > 0
-Basic integrity
-MIME validation

LAYER 2 — ADVANCED VALIDATION

-Resolution thresholds
-Aspect ratio checks
-Checking for corrupted bytes
-Sharpness and blur detection

LAYER 3 — HASHING & DUPLICATES

-Perceptual hashing
-Cross-dataset dedup
-Duplicate cluster reporting

LAYER 4 — SAFETY & COMPLIANCE

-NSFW classification
-Restricted content filtering
-Domain blacklists

2.VALIDATION REPORTS

EACH DATASET INCLUDES:

-Counts per phase
-Rejected items & reasons
-Duplicate sets
-Final accepted assets

STORAGE & PERFORMANCE OPTIMIZATION

1.MULTI-TIER STORAGE

PIXCRAWLER USES **HOT / WARM / COLD TIERING:**

-Hot (fast) → immediate downloads, recent datasets
-Warm (cheaper) → archived builds
-Cold (cheapest) → long-term retention

2. PERFORMANCE TECHNIQUES

-Parallel downloads
-Chunked streaming
-Redis caching
-CDN for fast public delivery
-Efficient file formats (WebP, JPEG XL optional)

3. CLEANUP POLICIES

-Expire temporary downloads
-Auto-delete failed pipeline outputs
-Scheduled cleanup jobs

SCALABILITY & INFRASTRUCTURE

1.HORIZONTAL SCALABILITY

-Celery workers scaled via containers
-Worker autoscaling
-Stateless API layer

2.ASYNC & PARALLEL PROCESSING

-Async FastAPI
-Task queues for heavy operations
-Distributed URL fetching

SCALABILITY & INFRASTRUCTURE

3.RELIABILITY

-Retries with exponential backoff
-Per-phase checkpoints
-Crash recovery via job state persistence

4.MONITORING

-Prometheus metrics
-Structured logs
-Job-level analytics

SECURITY & COMPLIANCE

1. AUTHENTICATION & AUTHORIZATION

-Supabase Auth (JWT)
-Role-based access control
-Tenant isolation

2.API SECURITY

-Rate limiting
-Input sanitization
-CORS restrictions
-HTTPS enforced

3. DATA REGULATIONS

-GDPR-friendly retention
-User data deletion
-Content compliance filtering

SECURITY & COMPLIANCE

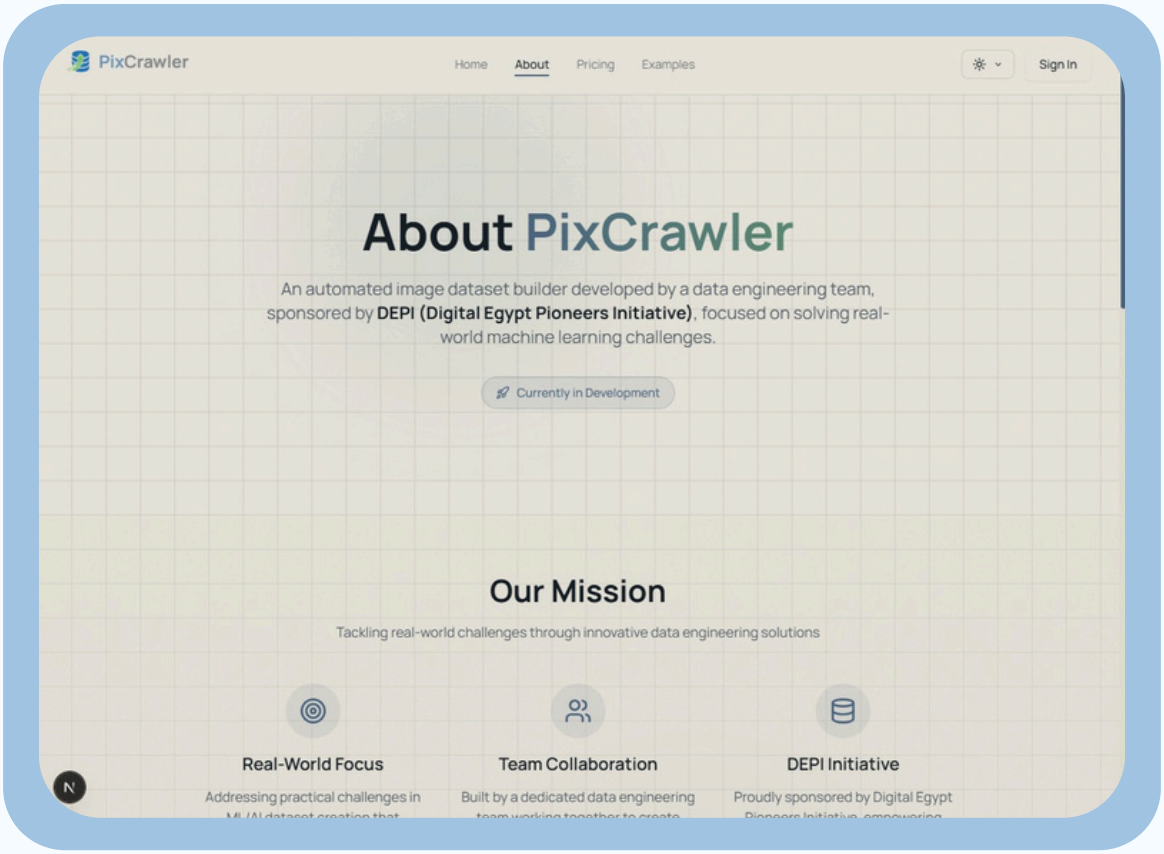
4.STORAGE SECURITY

-AES-256 at rest

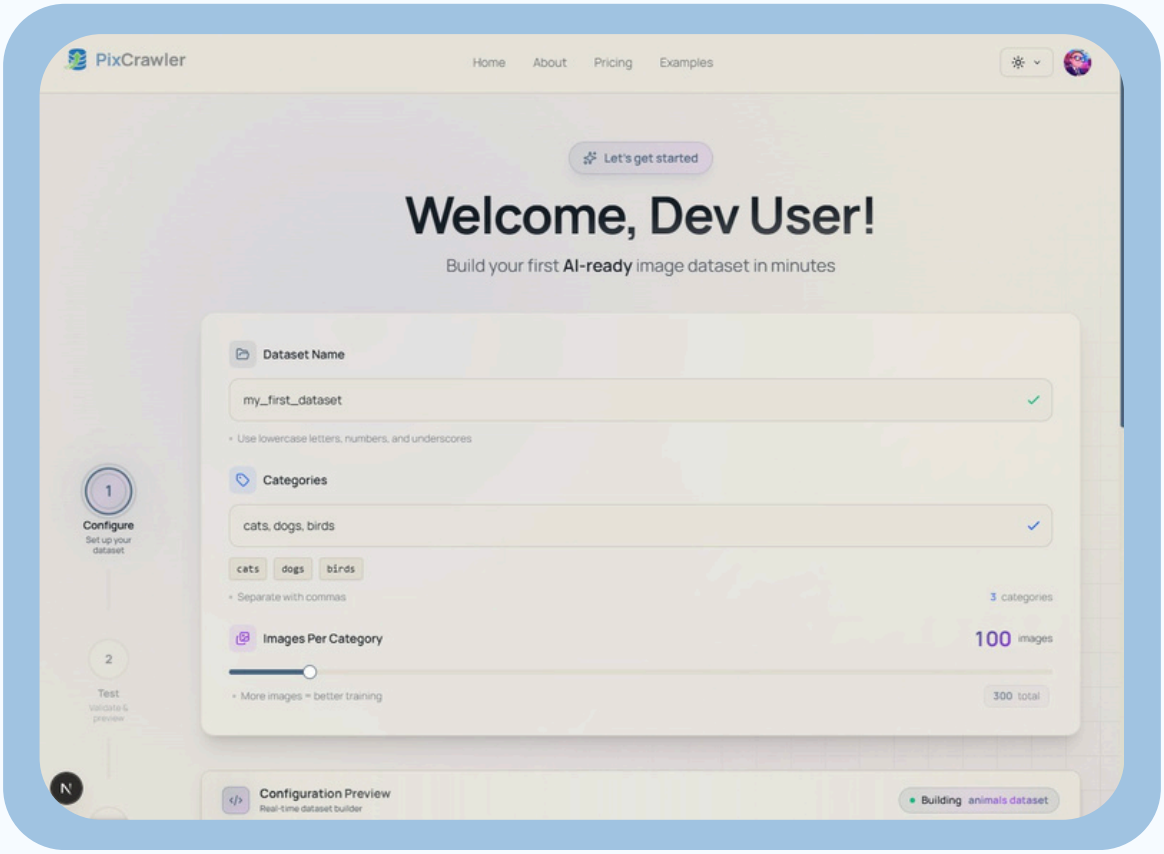
-Signed URLs

-Access logs on all downloads

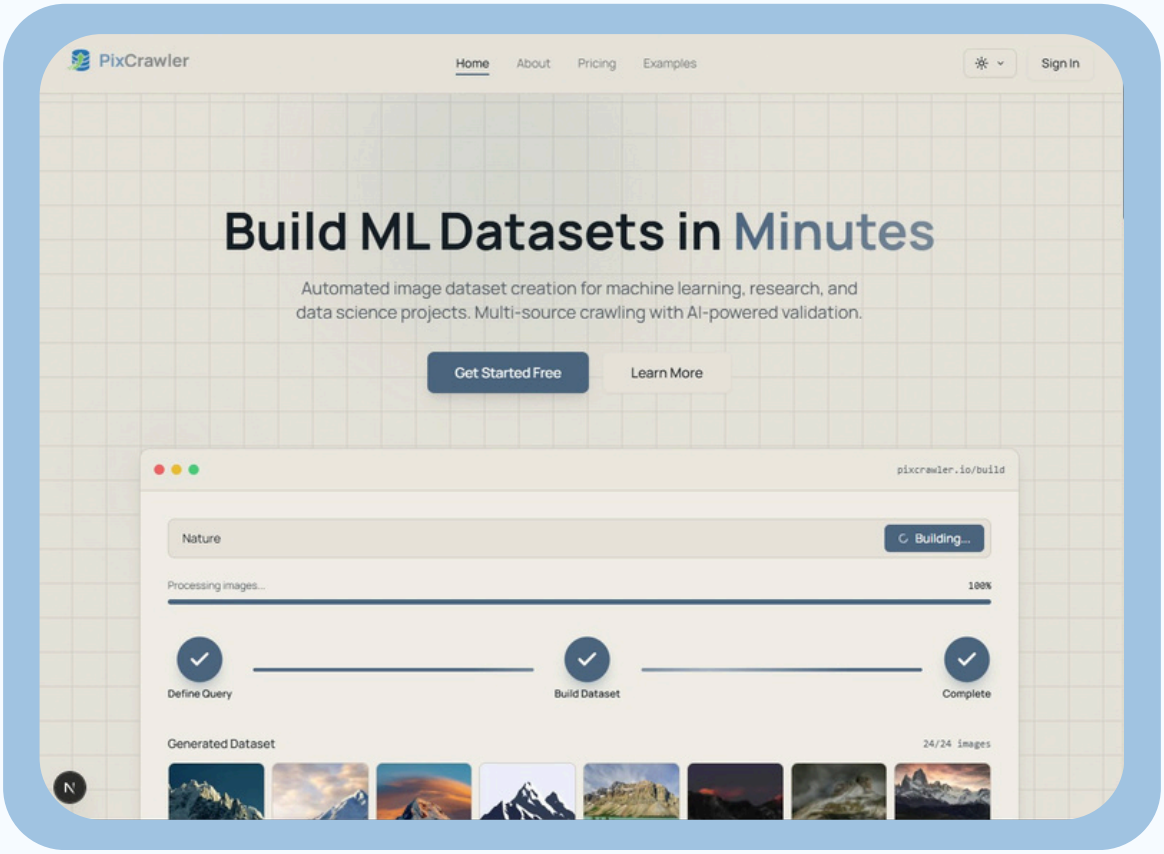
UX | UI DESIGN



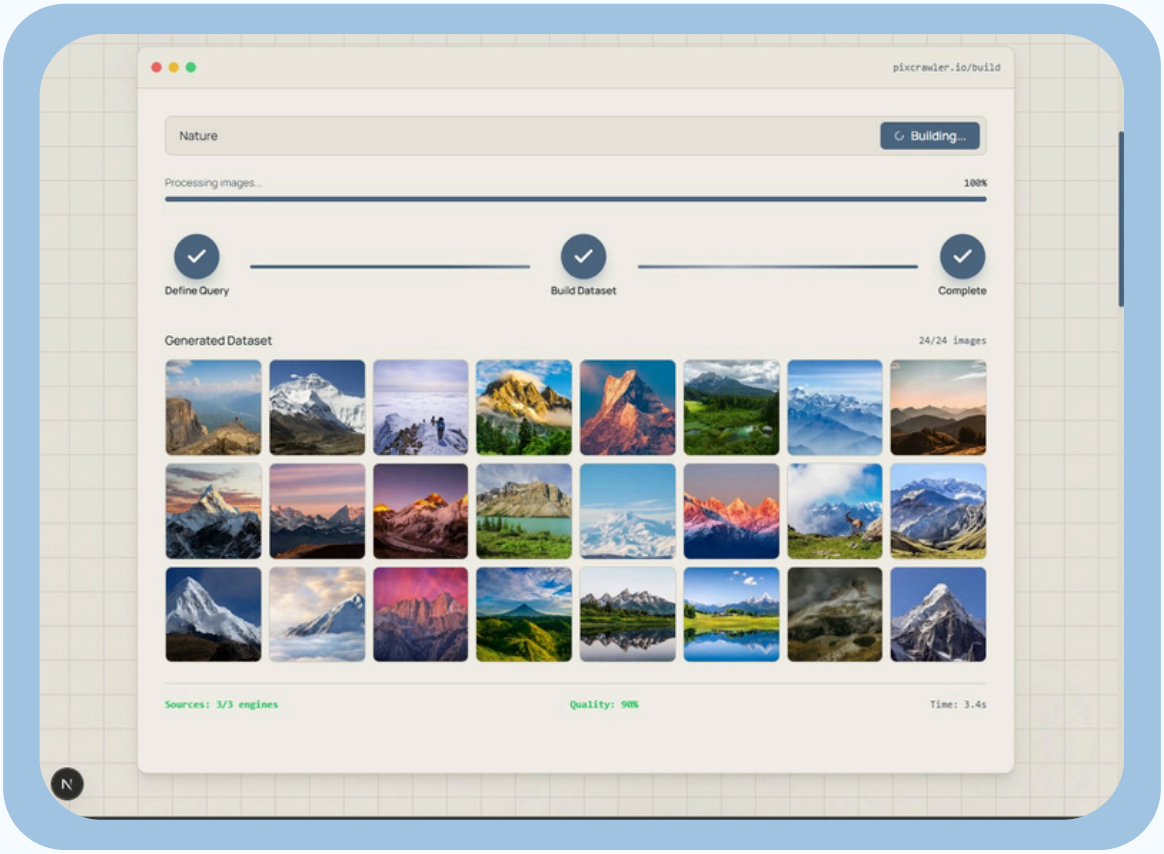
UX | UI DESIGN


PixCrawler
Home
About
Pricing
Examples
Let's get started
Welcome, Dev User!
Build your first AI-ready image dataset in minutes
Dataset Name
my_first_dataset
Use lowercase letters, numbers, and underscores
Categories
cats, dogs, birds
cats
dogs
birds
Separate with commas
3 categories
Images Per Category
100 images
More images = better training
300 total
1
Configure
Set up your dataset
2
Test
Configuration Preview
Real-time dataset builder
Building animals dataset

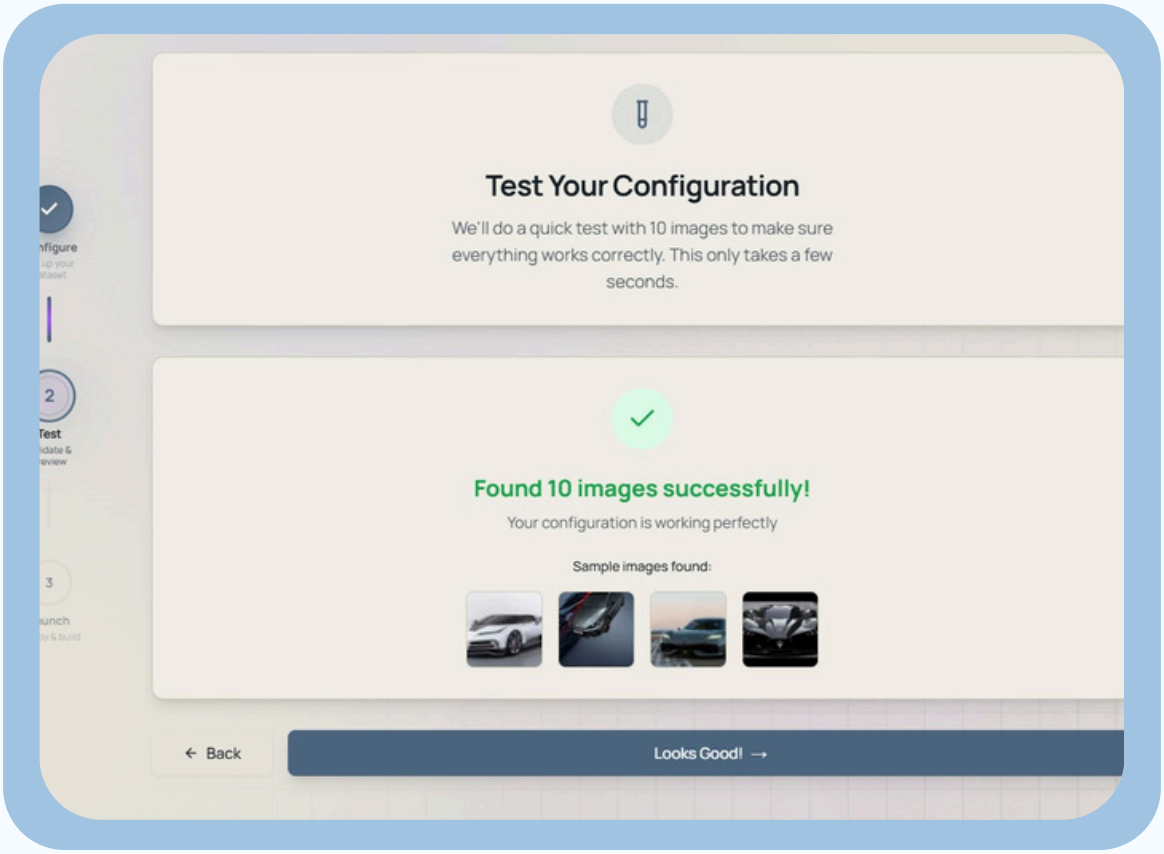
UX | UI DESIGN


PixCrawler
Home
About
Pricing
Examples
Sign In
Build ML Datasets in Minutes
Automated image dataset creation for machine learning, research, and data science projects. Multi-source crawling with AI-powered validation.
Get Started Free
Learn More
pixcrawler.io/build
Nature
Building...
Processing images...
100%
Define Query
Build Dataset
Complete
Generated Dataset
24/24 images

UX | UI DESIGN


pixcrawler.io/build
Nature
Building...
Processing images...
100%
Define Query
Build Dataset
Complete
Generated Dataset
24/24 images
Sources: 3/3 engines
Quality: 90%
Time: 3.4s

UX | UI DESIGN


Test Your Configuration
We'll do a quick test with 10 images to make sure everything works correctly. This only takes a few seconds.
Found 10 images successfully!
Your configuration is working perfectly
Sample images found:
← Back
Looks Good! →

CI/CD & DEVOPS

1.ENVIRONMENTS

-Dev
-Staging
-Production

2.CI PIPELINE

-Lint (flake8, black)
-Test (pytest)
-Type-check (mypy)
-Build docker image
-Auto-versioning

3.CD PIPELINE

-Deploy API to container environment
-Migrate database automatically
-Run worker replicas
-Health checks & rollbacks

3.CD PIPELINE

-Logs → Logtail/ELK
-Metrics → Prometheus
-Alerts → Slack/Email